Advanced Computer-Aided VLSI System Design
Homework 1: Lossy Bitmask Encoder with Low Power Design

TA: 高珩翔 r14943009@ntu.edu.tw **Due Tuesday, Mar. 31st, 13:59**

TA: 蔡岳峰 f12943014@ntu.edu.tw

Data Preparation

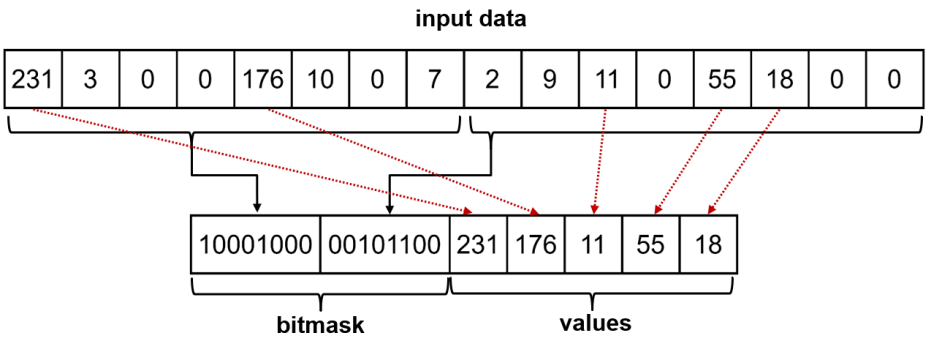
1. Unpack 1142_hw1.tar with the following command

```
tar -xvf 1142_hw1.tar
```

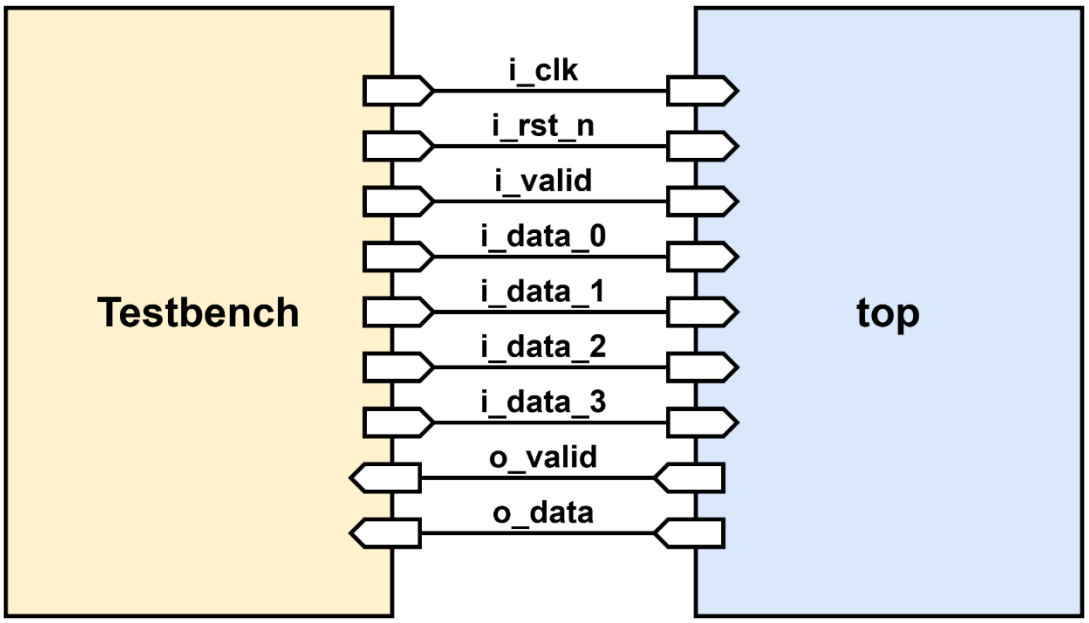
Folder	File	Description
00_TESTBED	testbench.v	File to test your design
00_TESTBED /p0~p4	pat*.dat	Input patterns
	gold*.dat	Output golden patterns
01_RTL	top.v	Your design
	rtl_01.f	File list for RTL simulation
	01_run	VCS command for simulation
	02_run_UPF	VCS command for simulation
02_GATE	03_gatesim	VCS command for simulation
	04_gatesim_UPF	VCS command for simulation
	rtl_02.f	File list for RTL simulation

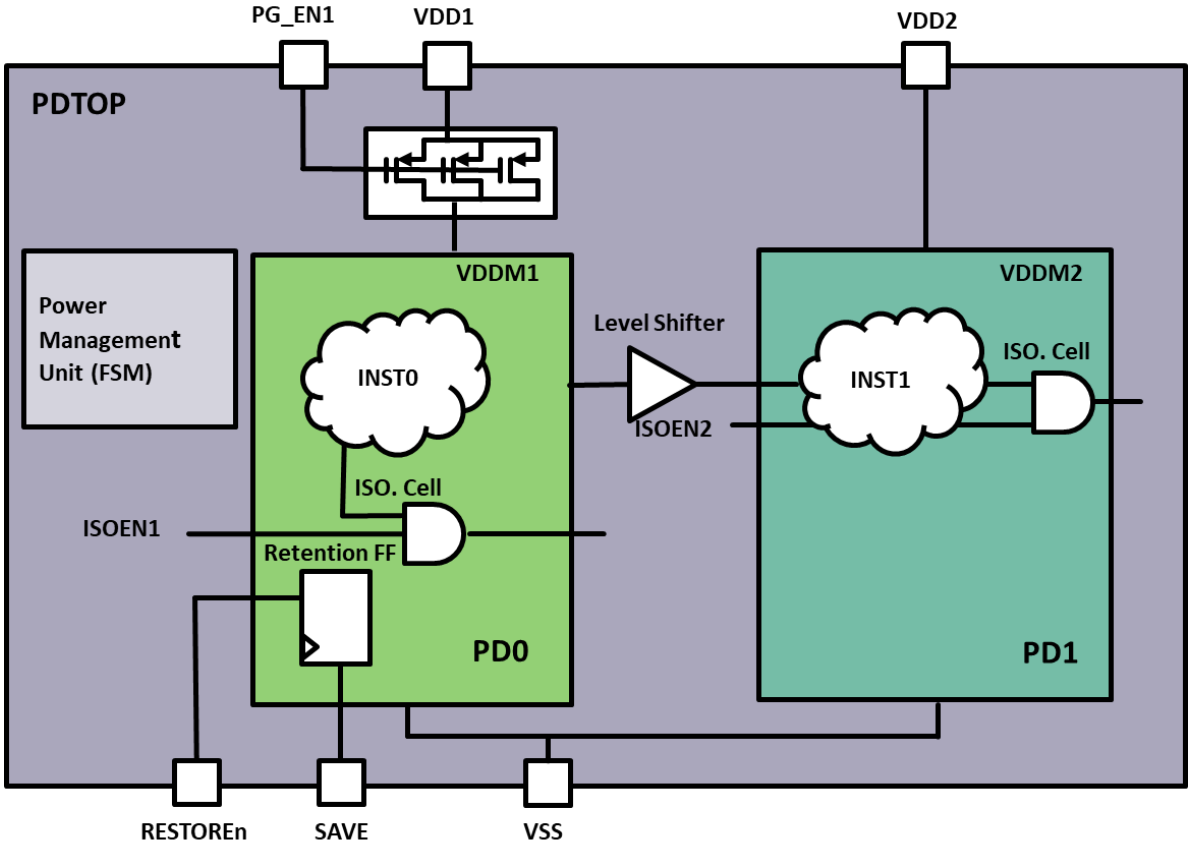
Introduction

Bitmask encoding is a form of data compression in which the position of non-zero elements are recorded with a bitmask, and only the non-zero values are stored in order. In this homework, you are going to design a lossy bitmask encoder, which treats small values as zeroes, and implement it with low-power design constraint through UPF flow.



Block Diagram





Specifications

1. Top module name: top
2. Input/output description:

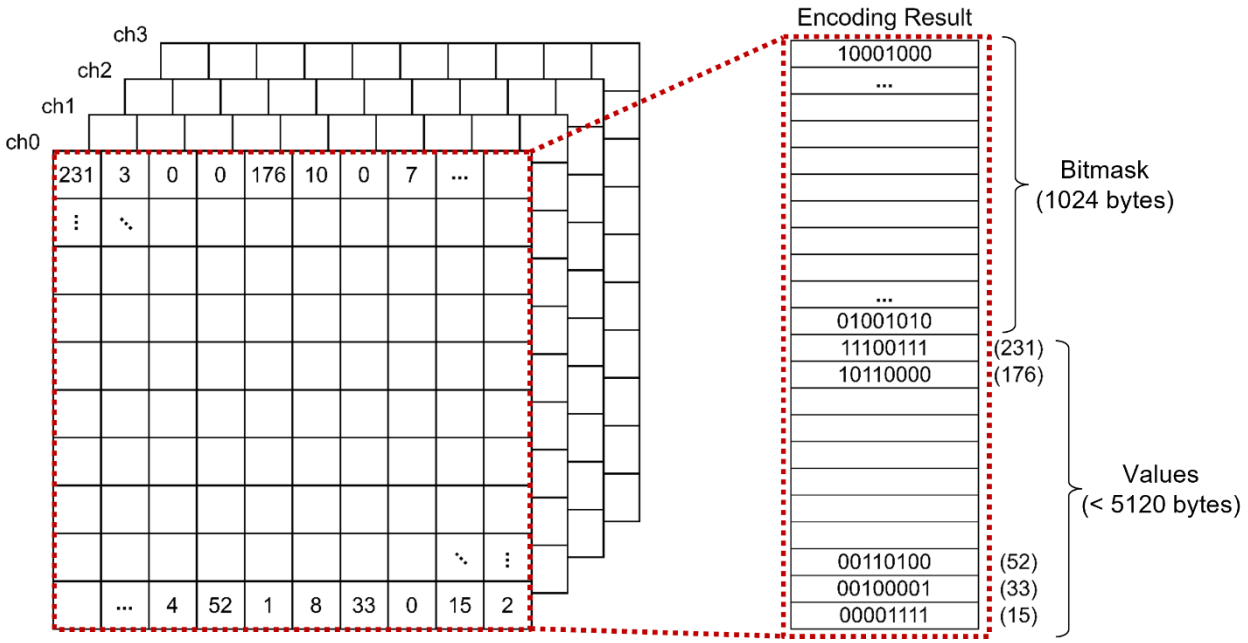
Signal Name	I/O	Width	Simple Description
i_clk	I	1	Clock signal in the system All inputs are synchronized with the negative edge All outputs should be synchronized with the positive edge
i_rst_n	I	1	Active **low** asynchronous reset
i_valid	I	1	The signal is **high** if input data is ready
i_data_0	I	8	Unsigned integer input data from 4 channels. Value range 0 ~ 255.
i_data_1	I	8	
i_data_2	I	8	
i_data_3	I	8	
o_valid	O	1	Set **high** if ready to output result, **raise only once**
o_data	O	8	Unsigned output data after bitmask encoding: 1. For the first 1024 cycles, output the bitmask (packed into bytes), then output the real values sequentially 2. Repeat for all 4 channels

3. Active low asynchronous reset is used only once.
4. All inputs are synchronized with the **negative** clock edge.
5. All outputs should be synchronized with the **positive** clock edge. That is, flip-flops should be added before all outputs.
6. The input pattern (i_data_0, i_data_1 and i_data_2, i_data_3) is ready only when i_valid is high.
7. o_valid should be high for only one cycle for each o_data.
8. The testbench will sample o_data at **negative** clock edge if o_valid is high.
9. You can raise o_valid at any moment **but only once.**

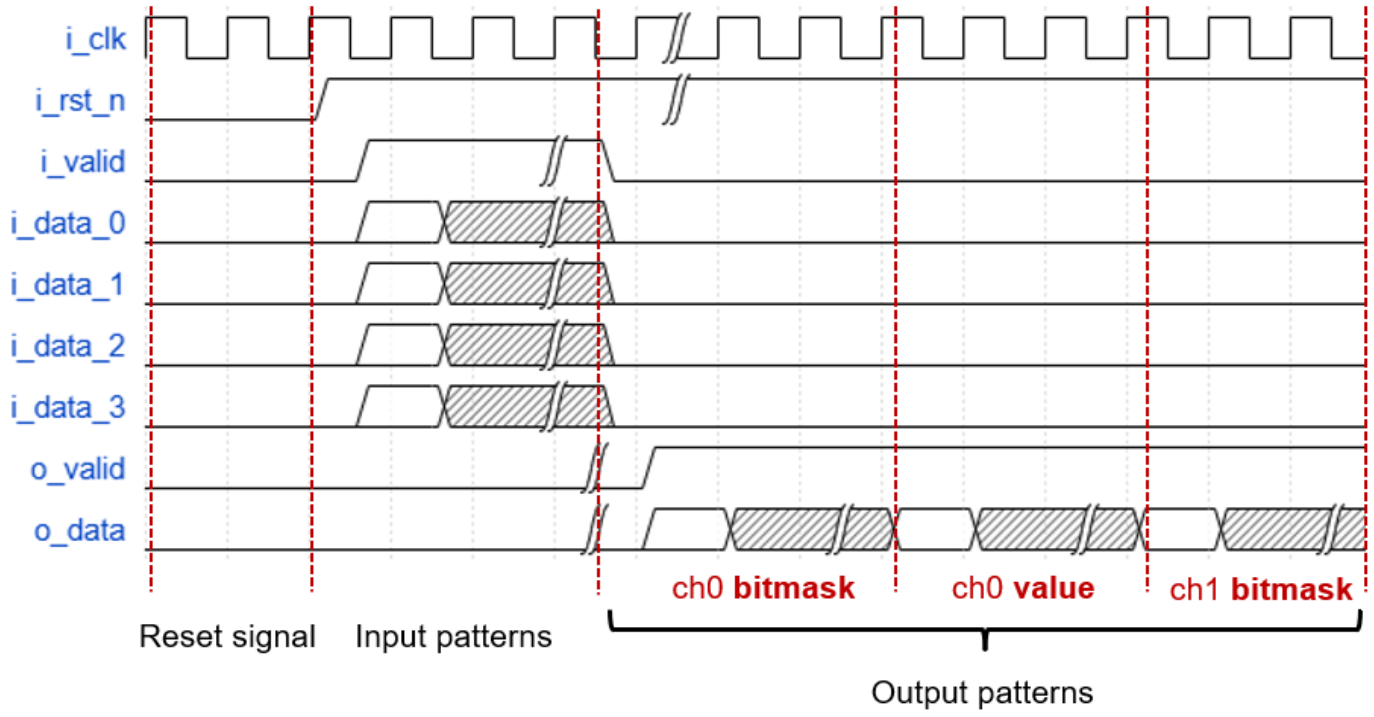
Design Description

1. Lossy Bitmask Encoding, values not exceeding the threshold are treated as zeroes, and will not be stored in the output value list.
2. The input length of each channel is **8192**.
3. Input data are separated by channel, while output data are serial with the bitmask and value list interleaved.
4. The length of the encoded value list in each channel won't exceed **5120**.

5. The range threshold value is set to **10**.
6. **At least one SRAM** should be implemented in your design.
7. At least **two power domains** (including top) are used in your design.
8. At least **one power switch** is used in your design.
9. Power top should be set in the UPF file.
10. You are **NOT** allowed to use DesignWare.

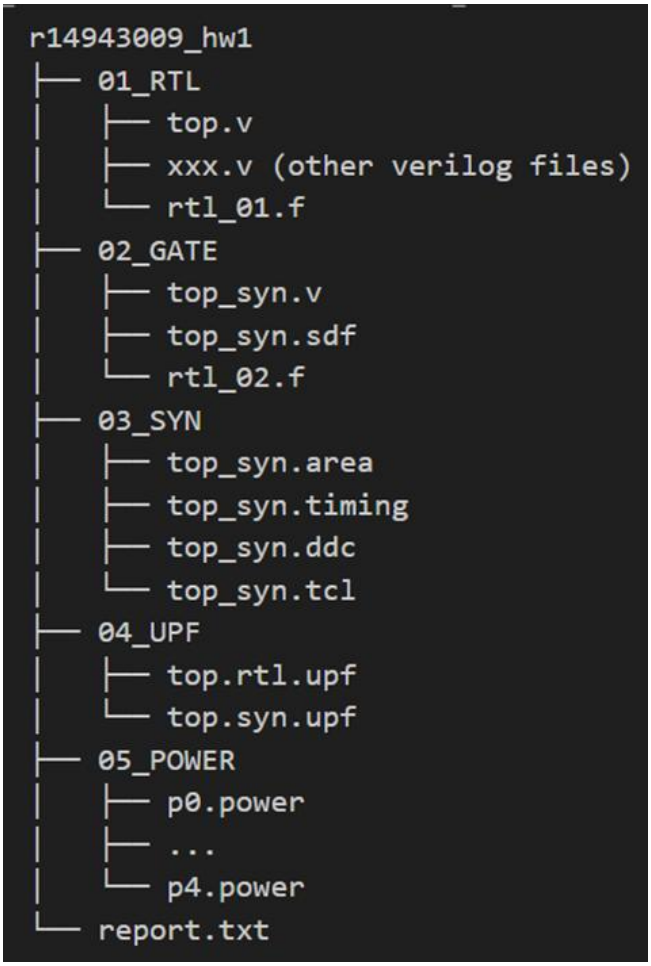


Sample Waveform



Submission

1. Create a folder named **studentID_hw1** and follow the hierarchy below.



Note: Use **lowercase** for all the letters. (e.g. r14943009_hw1)

2. Pack the folder **studentID_hw1** into a **tar.gz** file named **acvsd-xx-hw1.tar.gz**. TA will only check the last version after the homework deadline.

```
tar -zcvf acvsd-xx-hw1.tar.gz [path to studentID_hw1]          (ADFP)
```

```
tar -zcvf acvsd-xx-hw1-vk.tar.gz [path to studentID_hw1]   (NTU COOL)
```

Note:

a. Use **lowercase** for all the letters. (e.g. acvsd-11-hw1.tar.gz)

b. Remember to add *vk* (*k* is the number of version, *k* = 1, 2, …) for files submitted to NTU COOL. (e.g. acvsd-11-hw1-v1.tar.gz)

c. Pack the folder on the ADFP server to avoid OS related problems.

3. Submit to

i. NTU COOL

ii. Place the tar.gz file at the home directory of your ADFP account.

Grading Policy

1. TA will run your code with the following format of command. Make sure to run this command with no error message on the ADFP server.

   RTL simulation:

   ```
   vcs -full64 -R -f rtl_01.f +v2k -sverilog -debug_access+all
   +define+$1
   ```

   Gate-level simulation:

   ```
   vcs -full64 -R -f rtl_02.f +v2k -sverilog -v2005 -debug_access+all
   +maxdelays -negdelay +neg_tchk +define+$1+SDF
   ```

   For VCS NLP, additional arguments will be added

   ```
   -upf [path to UPF file] +define+UPF
   ```

2. Pass all the test pattern (5 public and 5 private) to get full score.
   - Simulation: **70%**

	Score
RTL simulation	15%
RTL with UPF	15%
Gate-level simulation	20%
Gate-level with UPF	20%

   - Performance: **30%**
     - Score for performance to be considered:

$$\textbf{Score} = \sum(P^2 \times T) \times A$$

P: Power (mW), T: Simulation time (ns), A: Area (um^2)

     - Only if you pass all patterns will you get the score.
     - Baseline = 1×10^7

	Score
Baseline	10%
Ranking (Need to pass baseline)	20%

3. Lose **5 points** for any incorrect naming or format.
   - It is your responsibility to ensure that the files can be correctly unpacked and executed on the ADFP server.
4. No late submission
   - 0 point for this homework
5. No plagiarism
   - Plagiarism in any form, including copying from online sources, is strictly prohibited.

References

1. Reference for bitmask encoding (If there is any conflict, the homework instructions shall prevail)
   Bitmask Encoding
2. Reference for UPF
   Synopsys ® Multivoltage Flow User Guide